

Agent Design Document

AI Productivity Agent

1. Problem Statement

Knowledge workers routinely manage tasks and meetings across fragmented tools such as calendars, to-do lists, and note-taking applications. Switching between these tools, manually structuring reminders, and re-reading meeting notes to identify action items impose a significant cognitive and administrative overhead. Traditional productivity software requires users to know exactly where information is stored and how to phrase commands within a rigid interface, rather than allowing users to express intent naturally.

The AI Productivity Agent addresses this gap by allowing users to manage their tasks and meetings entirely through natural language. Rather than relying on brittle keyword matching or regular-expression-based command parsing, the agent uses a locally hosted large language model (LLM) to interpret user intent, reason about the appropriate course of action, and invoke the correct backend tool. This reduces friction in daily planning, improves consistency of task tracking, and automatically surfaces actionable items from unstructured meeting notes.

2. Users

The agent is designed for individuals who manage a moderate-to-high volume of tasks and meetings and who value conversational, low-friction interaction over manual data entry.

User Group	Primary Needs
Students	Track assignment deadlines, exam schedules, and study sessions; extract action items from lecture or group-project notes.
Professionals	Manage daily tasks and recurring meetings; receive reminders; quickly query upcoming commitments.
Freelancers	Organise client-specific tasks and deadlines; log meeting outcomes; avoid missed deliverables across multiple clients.
Project Managers	Coordinate team meetings, extract and assign action items, and track completion status across projects.

3. Use Cases

The table below summarises the principal user requests the agent must support.

#	Use Case	Description
1	Create Task	User describes a new task in natural language; agent extracts title, due date, and priority.
2	Update Task	User modifies an existing task's details, such as due date or priority.
3	Edit Task Description	User revises the description or notes attached to a task.
4	Delete Task	User requests removal of a task; agent seeks approval before deletion.
5	Complete Task	User marks a task as finished.
6	Search Tasks	User queries tasks by keyword, status, priority, or due date.
7	Create Meeting	User schedules a meeting with participants, time, and agenda.
8	Manage Meeting	User reschedules, cancels, or updates meeting details.
9	Extract Action Items	Agent parses meeting notes and generates a list of follow-up tasks.
10	Schedule Reminder	User requests a reminder for a task or meeting at a specified time.
11	Ask Questions	User asks conversational questions about stored tasks or meetings.

#	Use Case	Description
12	Approve/Reject Action	User confirms or cancels a pending destructive operation.

4. Agent Responsibilities

The agent is responsible for the end-to-end handling of a user request, from interpretation to execution, within the following bounds:

- Reasoning: Interpret the semantic intent of free-form user input using the LLM, without relying on hard-coded keyword rules.
- Tool Selection: Choose the correct backend tool (or sequence of tools) based on reasoned intent, using LangGraph to orchestrate multi-step workflows.
- Task Management: Create, update, edit, delete, complete, and search tasks stored in the SQLite database.
- Meeting Management: Create and modify meetings, and extract structured action items from unstructured meeting notes.
- Approval Handling: Identify destructive or irreversible operations and request explicit user confirmation before execution.
- Response Generation: Produce clear, context-aware natural-language responses summarising actions taken or information retrieved.

5. Agent Boundaries

To ensure safe and predictable behaviour, the agent must not:

- Invent tasks, meetings, dates, or other factual details not present in the database or explicitly provided by the user.
- Bypass the approval workflow for destructive actions, even if the user's phrasing implies urgency.
- Expose internal system details, API keys, database credentials, or raw stack traces in user-facing responses.
- Execute actions outside the defined tool catalogue, including arbitrary code execution or unsupported external integrations.
- Retain or transmit user data beyond the local application scope defined by the system architecture.

6. Tool Catalogue

Tool Name	Purpose	Input	Output
create_task	Add a new task to the database.	Title, description, due date, priority	Created task object
update_task	Modify fields of an existing task.	Task ID, updated fields	Updated task object
delete_task	Remove a task (requires approval).	Task ID	Deletion confirmation
complete_task	Mark a task as completed.	Task ID	Updated task status
search_tasks	Query tasks by filters.	Keyword, status, date range, priority	List of matching tasks
create_meeting	Schedule a new meeting.	Title, time, participants, agenda	Created meeting object
update_meeting	Modify or cancel a meeting (requires approval for cancellation).	Meeting ID, updated fields	Updated meeting object
extract_action_items	Parse meeting notes into task candidates.	Raw meeting notes text	List of proposed tasks

Tool Name	Purpose	Input	Output
schedule_reminder	Attach a reminder to a task or meeting.	Item ID, reminder time	Created reminder object
query_knowledge	Answer natural-language questions about stored data.	User question	Natural-language answer

7. State Model

The agent maintains runtime state as a LangGraph state object, passed between graph nodes on each turn. This state includes:

- **Conversation History:** The sequence of prior user and agent turns, used to preserve context across multi-turn interactions.
- **Current Request:** The parsed representation of the user's most recent message, including inferred intent.
- **Pending Approvals:** A record of any destructive action awaiting user confirmation, including the target tool and parameters.
- **Tool Outputs:** Results returned by the most recently invoked tool(s), used to inform response generation.
- **Database References:** Identifiers (task IDs, meeting IDs) required to correctly scope subsequent operations.

8. Approval Model

Operations that irreversibly alter stored data require explicit user approval before execution; read-only and reversible operations do not.

Operation	Approval Required?
Create task	No
Update task	No
Complete task	No
Delete task	Yes
Bulk delete tasks	Yes
Create meeting	No
Cancel meeting	Yes
Search / query tasks	No
Extract action items (preview only)	No
Confirm extracted action items into tasks	Yes
Schedule reminder	No

9. Error Strategy

The agent adopts a fail-safe approach: it never silently executes an uncertain action and always communicates errors in plain language.

- **LLM Failures:** If the local Ollama model fails to respond or produces an unparseable output, the agent retries once, then informs the user that the request could not be understood.
- **Tool Failures:** If a tool invocation raises an exception, the agent surfaces a concise, user-friendly error message and does not retry destructive operations automatically.
- **Invalid Input:** Malformed or incomplete input (e.g., an ambiguous date) triggers a clarifying question rather than a guessed value.
- **Database Errors:** Connection or constraint errors are caught centrally, logged internally, and reported to the user as a generic failure without exposing internal details.

- **Missing Records:** Requests referencing a non-existent task or meeting return a clear notification rather than a fabricated result.
- **User Cancellations:** If a user declines a pending approval, the associated action is discarded and the state is reset without side effects.

10. Security Considerations

- **API Keys and Secrets:** The agent uses a locally hosted Ollama model, eliminating reliance on external API keys; any configuration secrets are stored in environment variables, never hard-coded or logged.
- **User Data:** All task and meeting data is stored locally in SQLite and is not transmitted to external services.
- **Database Access:** Access to the SQLite database is mediated exclusively through parameterised queries within defined tool functions, preventing SQL injection.
- **Permissions:** The agent operates with the minimum privileges required to read and write its own database file; no elevated system access is granted.
- **Input Validation:** All user-supplied fields (dates, titles, IDs) are validated and sanitised before being passed to tools or persisted.
- **Logging:** Logs capture operational events (tool calls, errors) but exclude sensitive personal content of tasks and meetings.
- **Privacy:** Because processing occurs on a local LLM and local database, user data does not leave the host machine, supporting a privacy-by-design posture.